Security evidence: Zoom App Marketplace review

Evidence package for **Time on the record**, mapped to Zoom's security review requirements.

Zoom requirement	Evidence
SSDLC	ssdlc.md
SAST results	evidence/sast-eslint-report.txt
DAST results	evidence/dast-zap-report.html (also .json)
Privacy Policy	public/privacy.html, served at `/privacy.html`
Policy 1 of 3: Vulnerability management procedures	vulnerability-management.md
Policy 2 of 3: Infrastructure/dependency management policy	dependency-management.md
Policy 3 of 3: Incident management and response policy	incident-response.md
Supporting: dependency scan (SCA)	evidence/sca-npm-audit.txt, with the pre-fix result in evidence/sca-npm-audit-before-remediation.txt

Current results (2026-10-17)

Scan	Tool	Result
SAST	ESLint 10.8.1 + eslint-plugin-security 4.0.1, over `server.js` and all `public/*.js`	**0 errors, 0 warnings**
SCA	npm audit (npm 11.17.0)	**0 vulnerabilities**
DAST	OWASP ZAP 2.17.0, spider + active scan of `http://localhost:3999/`	**0 High, 0 Medium, 0 Low.** 2 Informational alerts, both triaged below.

DAST informational alerts (triaged)

- **Information Disclosure – Suspicious Comments (2):** ZAP matched the words "user" and "from" in source comments in `app.js` and `zoom-sdk.js`. The comments describe code behavior and contain no secrets, credentials, or internal hostnames. Triaged as a false positive.
- **Modern Web Application (1):** an informational notice that the page uses JavaScript modules. No action needed.

DAST scope note

- ZAP's DOM-based XSS rule needs a browser driver; it was skipped because the bundled ChromeDriver did not match the installed Chrome version.
- As a compensating control, the client code was reviewed manually and is covered by SAST:
  - no URL or user input is written to the DOM as HTML (labels use `textContent`);
  - the CSP blocks inline and third-party script.
- Planned: update ZAP's Selenium add-on and re-run with the DOM-XSS rule included.

Findings fixed in this cycle

Finding	Source	Fix
High-severity advisories in axios, brace-expansion, proxy-addr	npm audit	Dependency upgrades (commit `f1e1684`)
No security headers; inline scripts/styles that would need `'unsafe-inline'`	Zoom requirements	HSTS, nosniff, Referrer-Policy, strict CSP; all scripts/styles moved to files (commit `c39dddb`)
Marker label written with `innerHTML` (XSS sink)	Code review	Built with `textContent` (commit `c39dddb`)
CSP missing `frame-ancestors` / anti-clickjacking (Medium)	ZAP	`frame-ancestors 'self' https://*.zoom.us https://*.zoom.com`
Express's 404 page replaced the CSP (Medium)	ZAP	Explicit 404 handler that keeps the headers
No SRI on Zoom SDK; cross-domain script (Medium, Low)	ZAP	SDK served from the app's own origin via the pinned `@zoom/appssdk@0.16.42` npm package; `script-src 'self'` only
OAuth tokens kept in a signed but unencrypted session cookie	Code review	Tokens are discarded after the exchange; only an `authorized` flag is kept

Reproducing

```
npm run security-check
```

This runs SAST and `npm audit`. For DAST, see ssdlc.md §4.

Before submitting

1. Make sure the policies are true:
   - MFA is on for your Zoom, GitHub and hosting accounts;

    - Dependabot alerts are on for the GitHub repository;
    - production runs on a cloud host, not this PC's Cloudflare tunnel.
2. Run a DAST scan against your **production HTTPS URL** too, if you can. This report scanned a local instance, so it can't confirm production TLS settings.
3. Export the Markdown files to PDF if the submission form requires PDFs.
